



Template

1. Introduction and Goals	2
1.1 Requirements Overview	2
1.2 Quality Goals	2
1.3 Stakeholders.....	3
2. Architecture Constraints.....	4
2.1 Technical Constraints:.....	4
2.2 Organizational Constraints:	5
3. Context and Scope	5
3.1 Business Context:	5
3.2 Technical Context:.....	6
4. Solution Strategy.....	6
5. Building Block View	7
5.1 Whitebox Overall System	7
5.2 Level 2 -Application Server Internal Detail:	9
5.3 Level 3 -Whitebox enrollment-service	11
6. Runtime View	12
7. Deployment View.....	16
8. Cross-cutting Concepts.....	21
9. Architecture Decisions.....	25
10. Quality Requirements	28
10.1 Quality Requirements Overview	28
10.2 Quality Scenarios	29
11. Risks and Technical Debts.....	30
12. Glossary	31

1. Introduction and Goals

The University Management System (UMS) is a web-based platform developed for a regional Austrian university. The system was initiated to replace time-consuming manual administrative processes - including paper-based course enrollment, grade management and fee collection-with a centralized, reliable and secure digital solution.

1.1 Requirements Overview

- Requirements were identified through stakeholder interviews, workshops and analysis of existing university workflows.
- The following functional requirements were derived:

Requirement	Description
Course Management	Create, update and manage university courses and their capacity
Student Enrollment	Students enroll in courses - system enforces seat limits atomically
Grade Management	Lecturers enter and publish student grades
Exam Registration	Students register for exams - capacity limits strictly enforced
User Authentication	Role-based login for students, lecturers and administrators
Fee Management	Students view and pay university fees via integrated payment provider
Notifications	Automatic email notifications for grade publications and enrollment confirmations
Library Catalog	Students browse available books and resources
Announcements	University-wide news and announcements
External Integration	Integration with Moodle LMS, SendGrid email service and Stripe payment provider

- **Non-functional requirements** were made explicit through quality scenarios rather than vague statements - for example, not "the system must be fast" but "95% of requests must return results within 2 seconds under normal load."

1.2 Quality Goals

- The following five quality goals are defined according to the ISO 25010:2023 standard. They directly influenced the fundamental architectural decisions of the UMS and are ordered by priority.

Priority	ISO 25010 Category	Quality Goal	Concrete Scenario
1	Reliability	System operates without interruption during critical academic periods	500 students access exam registration simultaneously -system remains stable, no data loss, no duplicate registrations occur
2	Security	Personal and academic data is protected against unauthorized access	A student attempts to access a lecturer's grade page -the system rejects the request immediately based on role validation. All data is transmitted via HTTPS
3	Functional Suitability	All functions meet the stated needs of students, lecturers and administrators	A lecturer publishes a grade -the student sees the correct, updated grade immediately on all devices
4	Maintainability	Each service can be updated and redeployed without affecting other parts of the system	A bug is found in the grade service -the team fixes and redeploys only that service. No other service is restarted or affected
5	Performance Efficiency	System responds within acceptable time under normal and peak load	Under normal load the system responds within 2 seconds. During enrollment peak the enrollment service scales independently without slowing other services

1.3 Stakeholders

- The following stakeholders directly influence or are affected by the architecture of the University Management System.

Role	Contact	Expectations
Student	End user -web browser	Reliable enrollment, correct grades, exam registration -data must always be accurate
Lecturer / Professor	End user -web browser	Consistent grade entry and course management -no data loss or duplication
University Administrator	End user -web browser	Full user management, fee oversight and system monitoring
Software Architect	Development team	Clear architectural documentation, clean separation

		of concerns, justified decisions
Database Administrator	Operations team	Schema isolation per service, referential integrity, GDPR-compliant data handling
Moodle LMS	api.moodle.university.ac.at	Stable REST API integration for course material synchronization
SendGrid	api.sendgrid.com	REST API calls for email delivery -no direct SMTP connection
Stripe	api.stripe.com	Secure HTTPS payment API -PCI-DSS compliance required

2. Architecture Constraints

Constraints are requirements that limit architectural design choices. They were identified during requirements engineering and are treated as fixed boundaries -not open for discussion.

2.1 Technical Constraints:

ID	Constraint	Motivation
TC_1	PostgreSQL as the primary database	Required by project specification -no alternative permitted
TC_2	Java Spring Boot as backend framework	Team has existing Java expertise -required by project specification
TC_3	Angular as frontend framework	Required by project specification
TC_4	REST API for all service communication	Ensures loose coupling and standard interoperability between services and external systems
TC_5	GDPR compliance	EU legal requirement -all personal data must be encrypted, auditable and anonymizable
TC_6	Schema-per-Service pattern	Each microservice owns its own database schema -no direct cross-schema access permitted
TC_7	Each service must be independently deployable	Supports microservice architecture principle of independent deployment and scaling

2.2 Organizational Constraints:

ID	Constraint	Motivation
OC_1	Regional university scope	System is designed for one university, not a multi-tenant platform
OC_2	External services not self-hosted	Email, payment and LMS are operated by external providers - integration only via API
OC_3	Agile development process using SCRUM	Requirements are refined iteratively - not all requirements are defined upfront

3. Context and Scope

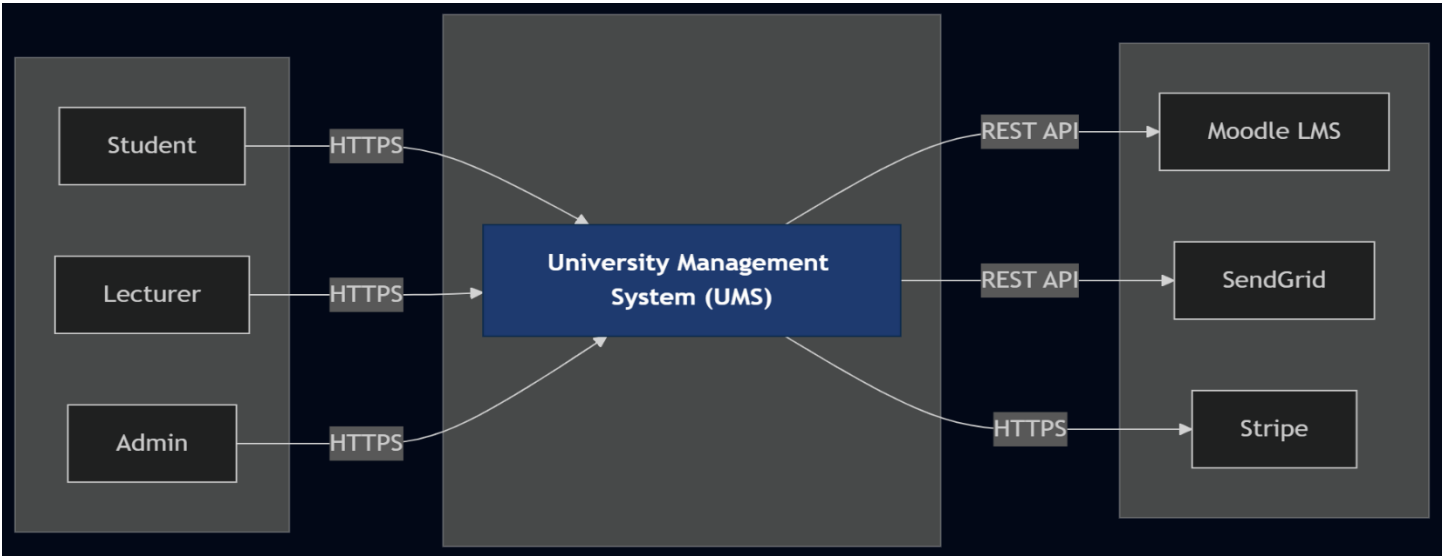
The University Management System is the central digital platform for all university administrative processes. The system boundary covers everything running on the university's own infrastructure. External systems -Moodle, SendGrid and Stripe - are outside the system boundary and are accessed only through standardized interfaces.

3.1 Business Context:

Communication Partner	Input to System	Output from System
Student	Login, course enrollment, exam registration, fee payment	Grades, confirmations, notifications, timetable
Lecturer	Grade entry, course management	Grade confirmation, course reports
Administrator	User management, fee management	System reports, user confirmations
Moodle LMS	Course material requests	Course access data via REST API
SendGrid	—	Email notifications - grade published, enrollment confirmed
Stripe	Payment initiation	Payment confirmation, transaction status

3.2 Technical Context:

Channel	Protocol	Used For
Browser ↔ Web Server	HTTPS (TLS 1.2+)	All user interactions
Web Server ↔ Application Server	REST API (HTTP/JSON)	Business logic requests
Application Server ↔ Database	JDBC/SQL	Data read and write operations
Application Server ↔ SendGrid	REST API (HTTPS)	Email delivery
Application Server ↔ Stripe	HTTPS	Payment processing
Application Server ↔ Moodle	REST API (HTTPS)	Course material access



4. Solution Strategy

The solution strategy defines the fundamental decisions that shape the architecture. Each decision is justified by a quality goal or constraint.

Decision	Solution	Justified By
Architecture style	Microservices with REST API	Allows independent scaling and deployment per service -addresses TC_7 and scalability goal

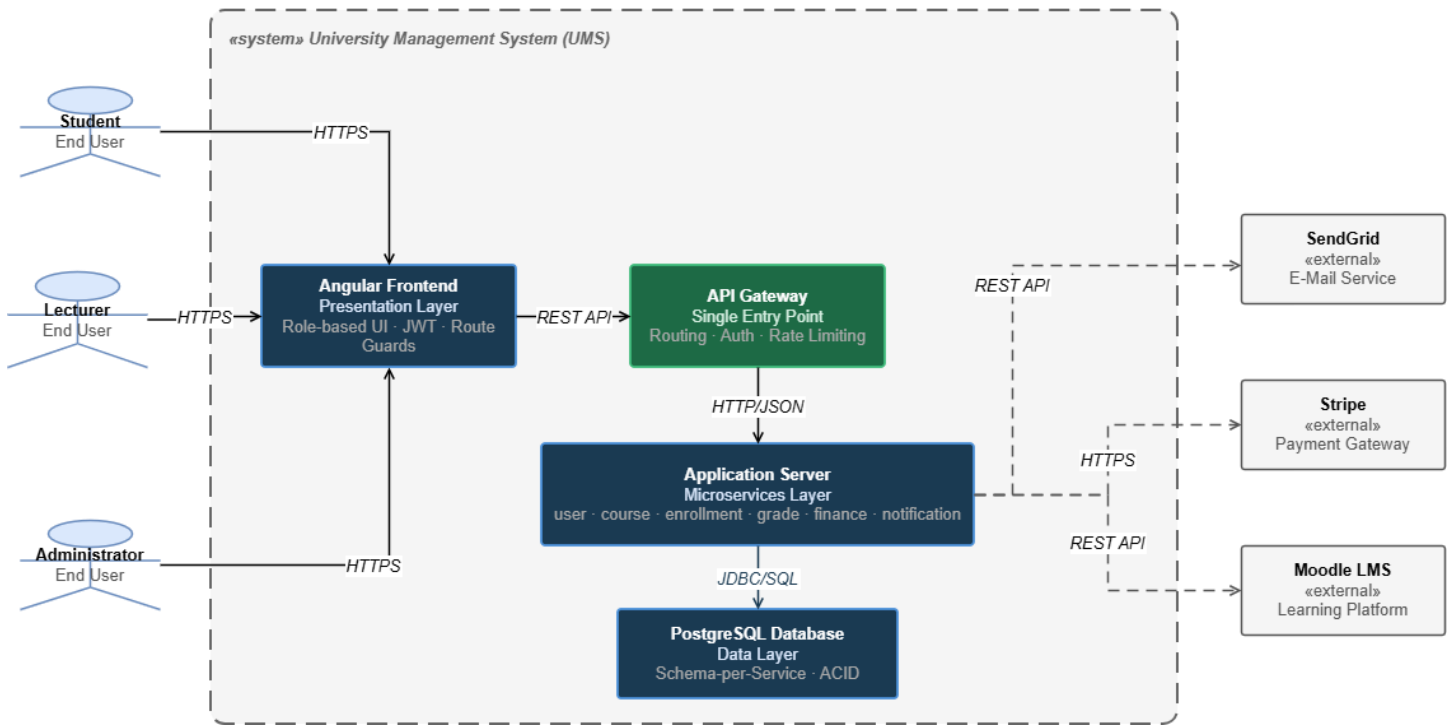
CAP strategy	CP -Consistency + Partition Tolerance	Grades and enrollments are legally binding - consistency is mandatory
Locking strategy	Pessimistic locking for grades, optimistic locking for enrollments	Grades require absolute correctness - enrollments require performance under high concurrency
Frontend framework	Angular	Role-based routing, reactive forms, JWT interceptor -addresses TC_3
Backend framework	Java Spring Boot	Transactional guarantees, Spring Security, team expertise -addresses TC_2
Database	PostgreSQL with Schema-per-Service	ACID transactions, GDPR compliance via pgcrypto, schema isolation - addresses TC_1 and TC_6
Email delivery	SendGrid REST API	Professional cloud provider - no SMTP server required, addresses OC_2
Payment	Stripe API	PCI-DSS certified -legal requirement for payment processing
Authentication	JWT tokens with BCrypt	Stateless, scalable, role-based access control across all services
Technical debt management	10-20% of each sprint allocated to debt reduction	Prevents accumulation of shortcuts that reduce long-term maintainability

The system evolved from a layered monolithic approach toward a microservice architecture to address the limitations of monoliths - specifically the inability to scale or deploy individual features independently. Unlike SOA which relies on shared services and central orchestration, our microservices are fully self-contained and communicate only through lightweight REST APIs - following the principle of high cohesion and low coupling.

5. Building Block View

5.1 Whitebox Overall System

The University Management System is decomposed into three tiers following the three-tier architecture pattern. The presentation tier handles all user interaction, the application tier contains all business logic as independent microservices, and the data tier provides persistent storage with strict schema isolation per service.



Contained Building Blocks:

Name	Responsibility
Angular Frontend	Presentation layer -serves role-specific UI to students, lecturers and administrators
API Gateway	Single entry point -handles routing, authentication and rate limiting for all services
user-service	Manages user accounts, roles and authentication -issues JWT tokens on login
course-service	Manages course catalog, capacity and scheduling
enrollment-service	Handles course enrollment with atomic seat reservation -prevents double booking
grade-service	Manages grade entry and exam results -pessimistic locking ensures consistency
finance-service	Manages student fees and payment processing via Stripe
notification-service	Sends email notifications asynchronously via SendGrid -does not block other services

PostgreSQL Database	Persistent storage -one schema per service, no cross-schema access
SendGrid	External email delivery -connected via REST API
Stripe	External payment processing -connected via HTTPS
Moodle LMS	External learning platform -connected via REST API

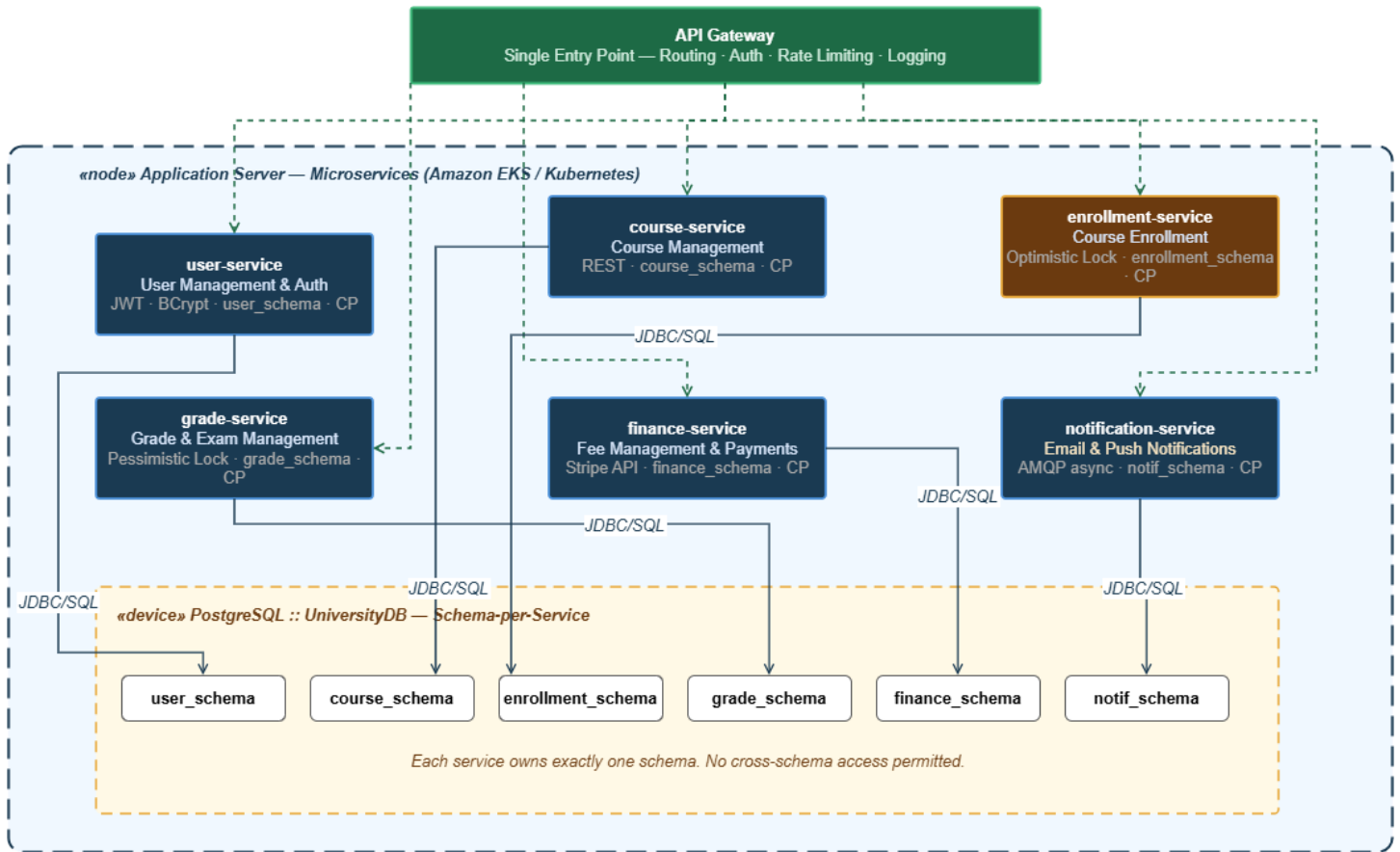
Important Interfaces:

Interface	From	To	Protocol
User interaction	Browser	Angular Frontend	HTTPS
API requests	Angular Frontend	API Gateway	HTTPS/JSON
Service routing	API Gateway	All microservices	HTTP/JSON
Database access	Each microservice	Own schema in PostgreSQL	JDBC/SQL
Email delivery	notification-service	SendGrid	REST API
Payment	finance-service	Stripe	HTTPS
Course sync	course-service	Moodle LMS	REST API

5.2 Level 2 -Application Server Internal Detail:

Each microservice follows the layered pattern internally: Controller → Service → Repository. Services never call each other directly -all inter-service communication goes through the API Gateway. This enforces low coupling and allows each service to be deployed, scaled and updated independently.

Building Block View — Level 2: Whitebox Application Server

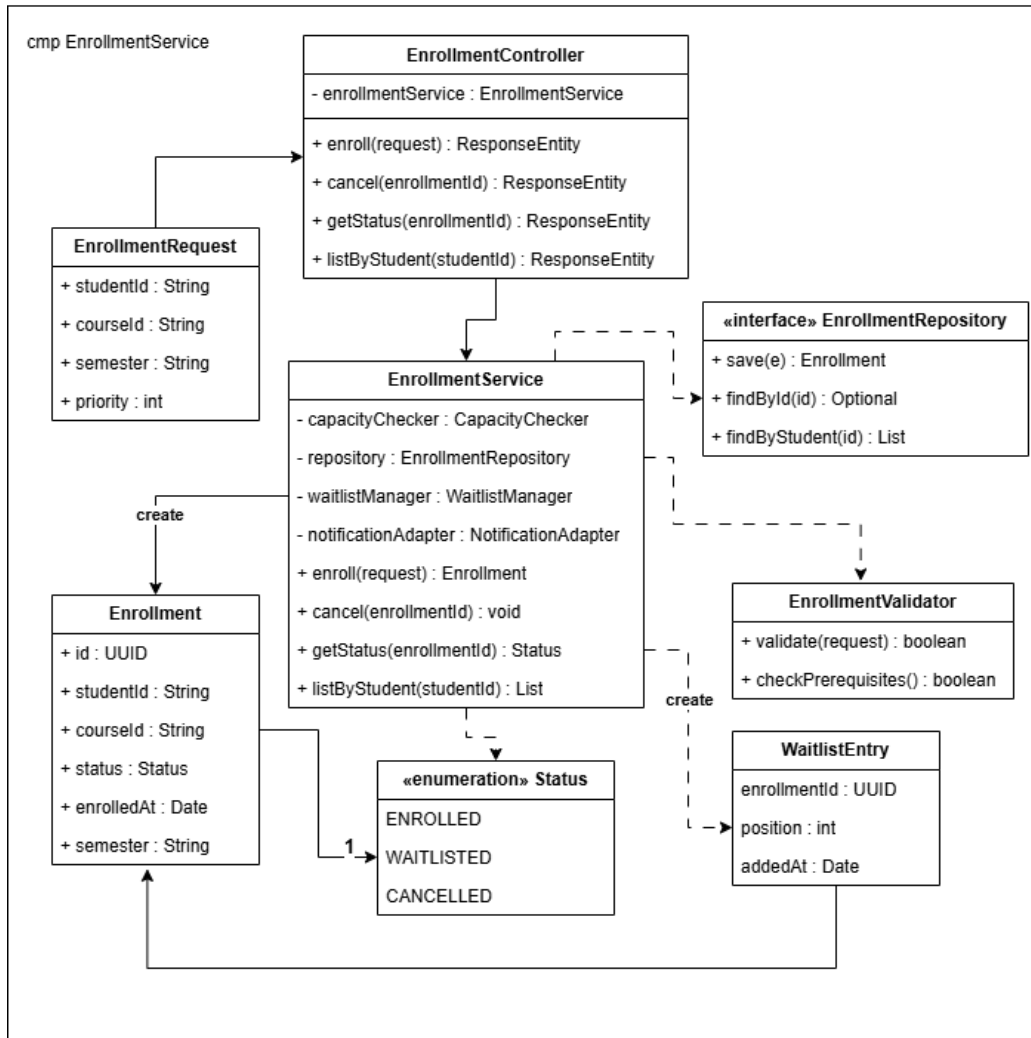


Service	Key Behavior	CAP	Locking Strategy
user-service	User management and JWT token issuance on login - BCrypt password hashing	CP	Transaction-based
course-service	Course catalog management - capacity and scheduling	CP	Optimistic locking with version numbers
enrollment-service	Atomic seat reservation - one seat, one student, no double booking	CP	Optimistic locking with @Version
grade-service	Grade and exam management - strict write control	CP	Pessimistic locking - row lock during update
finance-service	Fee management and payment processing via Stripe API	CP	Pessimistic locking - payment must be atomic
notification-service	Asynchronous email and push notifications via AMQP	CP	No locking needed

5.3 Level 3 -Whitebox enrollment-service

The enrollment-service handles course enrollment with strict capacity enforcement. It implements CP - Consistency + Partition Tolerance -through optimistic locking to prevent double-booking.

Level 3 – EnrollmentService (core domain, white box)



- Contained Components:

Component	Responsibility
EnrollmentController	Exposes REST endpoints -delegates all requests to EnrollmentService
EnrollmentService	Core business logic -orchestrates enrollment, cancellation and status checks
EnrollmentRepository	Data access -persists and retrieves Enrollment entities
EnrollmentValidator	Validates requests and checks student prerequisites
CapacityChecker	Checks whether a course has available seats
WaitlistManager	Manages waitlist positions when a course is full
NotificationAdapter	Sends notifications on status change -decoupled via adapter pattern

- Key Design Decisions:

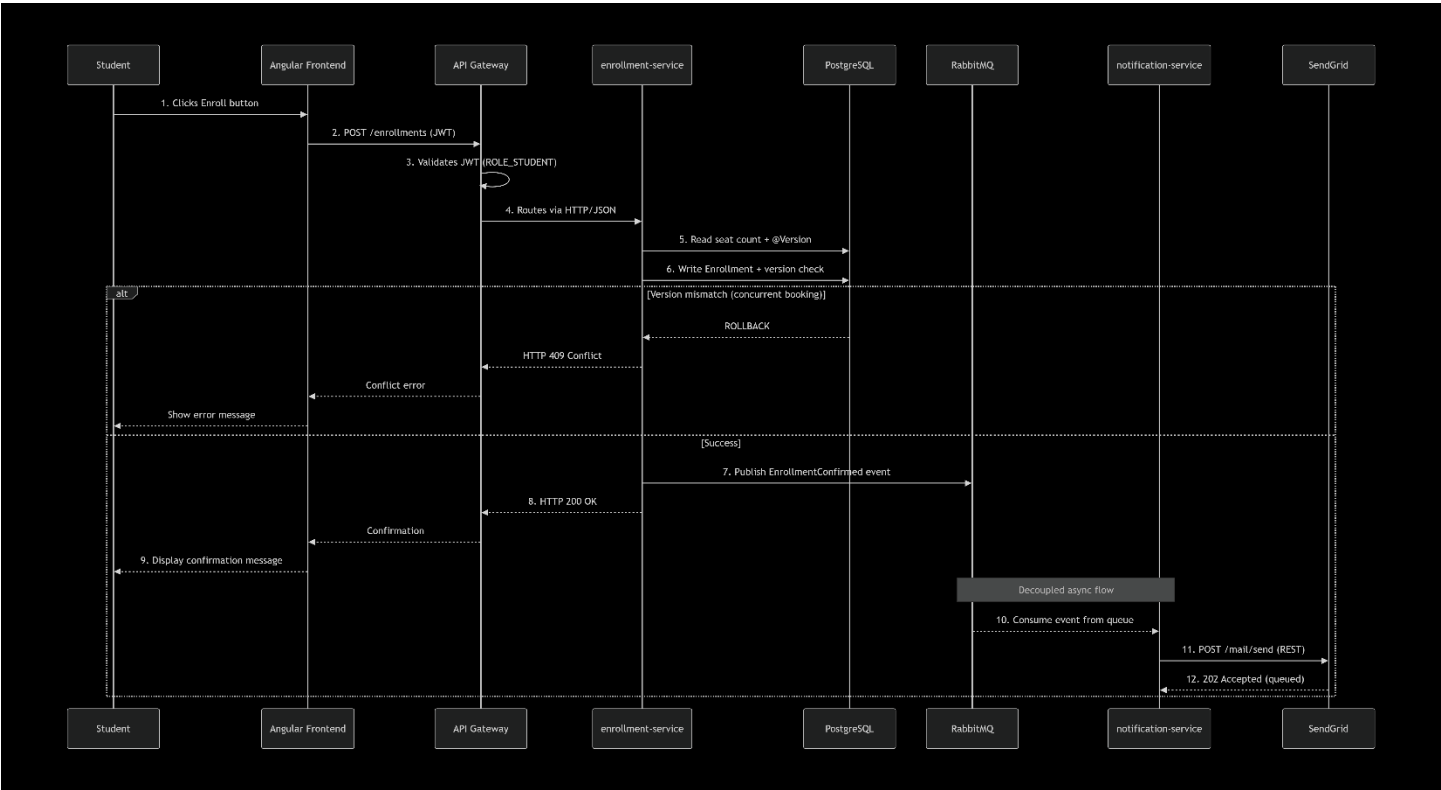
Decision	Reason
Optimistic Locking (@Version)	Prevents two students booking the last seat simultaneously
@Transactional on enroll()	Atomic write -enrollment either fully succeeds or fully fails

Adapter Pattern for notifications	If notification fails, enrollment still succeeds -no dependency
Waitlist Pattern	Fair first-come-first-served seat promotion when cancellations occur

6. Runtime View

The runtime view describes how the building blocks of the University Management System interact at runtime. Two scenarios are documented -the most architecturally relevant use case (course enrollment) and a critical security scenario (user login).

- **Runtime Scenario 1 - Course Enrollment**
- This scenario is the most architecturally relevant because it demonstrates optimistic locking, capacity enforcement, asynchronous notification and the interaction between multiple services simultaneously.



- **What the diagram shows:**
- The request flows from the student's browser through five system components before returning a confirmation. The notification is deliberately decoupled from the enrollment transaction -the student receives a response before the email is sent.

Step-by-step description:

Step	From	To	Action
1	Student	Angular	Student clicks Enroll button in the course catalog

		Frontend	
2	Angular Frontend	API Gateway	Sends POST /enrollments with JWT token via HTTPS
3	API Gateway	API Gateway	Validates JWT -confirms role is ROLE_STUDENT
4	API Gateway	enrollment-service	Routes the request via HTTP/JSON
5	enrollment-service	PostgreSQL	Reads current seat count and @Version field from course
6	enrollment-service	PostgreSQL	Writes new Enrollment record -PostgreSQL checks version and commits atomically
alt	—	—	If version mismatch (another student booked simultaneously) → ROLLBACK → HTTP 409 Conflict
7	enrollment-service	RabbitMQ	Publishes EnrollmentConfirmed event asynchronously -does not wait for email delivery
8	enrollment-service	Angular Frontend	Returns HTTP 200 OK -enrollment confirmed
9	Angular Frontend	Student	Displays confirmation message
10	notification-service	RabbitMQ	Consumes event from queue independently
11	notification-service	SendGrid	Calls POST /mail/send via REST API
12	SendGrid	notification-service	Returns 202 Accepted -email queued for delivery

Notable architectural aspects:

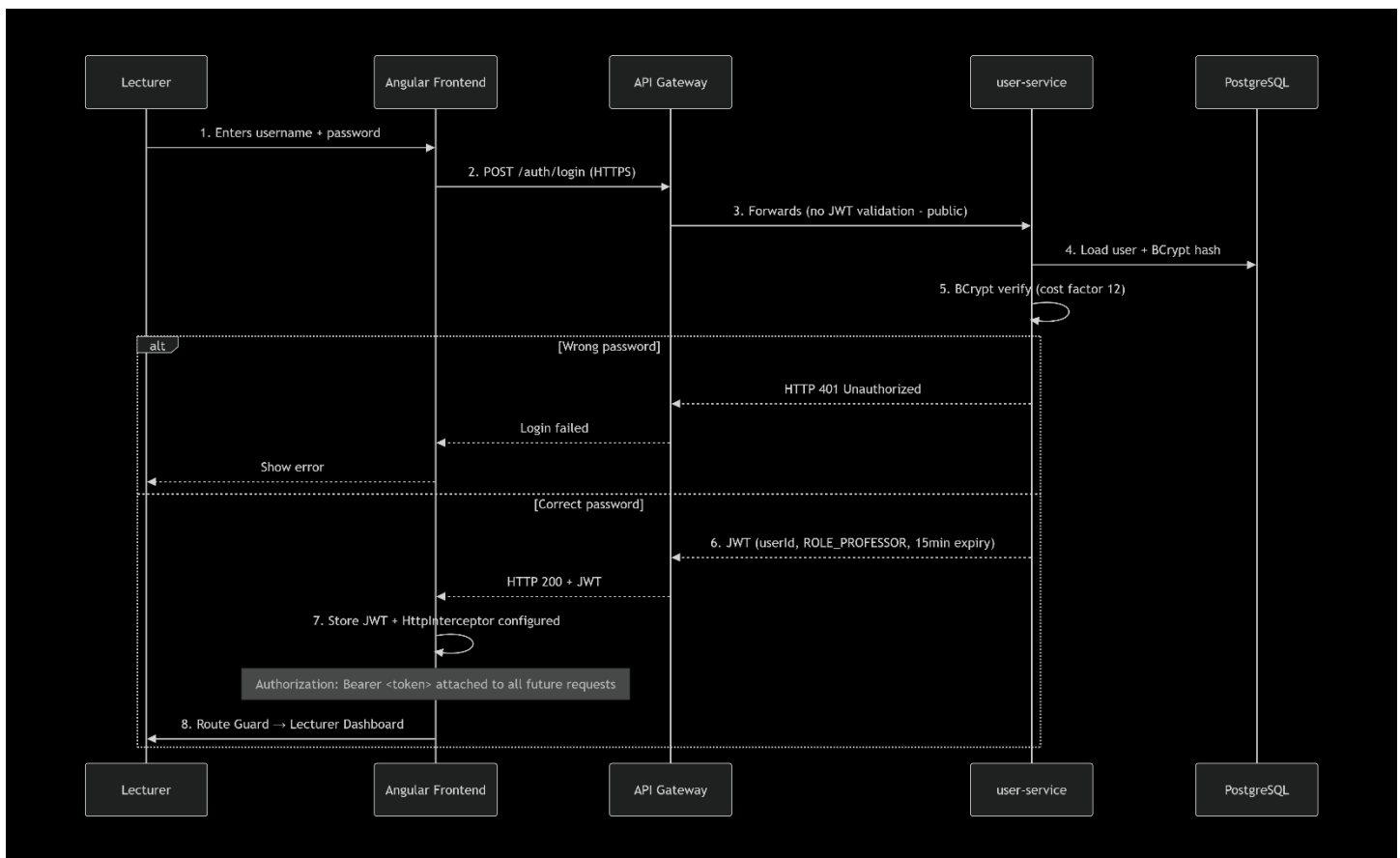
- **Steps 5–6 -Optimistic Locking:** The @Version field on the Course entity prevents two students from booking the last seat simultaneously. No database lock is held during the entire request -only at commit time. If the version has changed, the transaction rolls back and the student receives a clear conflict error.
- **Steps 7–12 -Choreography Pattern:** The notification is fully decoupled from the enrollment. The enrollment transaction completes and returns success to the student before the email is sent. If

SendGrid is temporarily unavailable, the event remains in the RabbitMQ queue and is delivered when the service recovers -the enrollment is never affected.

- **Step 3 -Centralized Authentication:** The API Gateway is the single point of JWT validation. The enrollment-service never needs to validate tokens itself -this is handled once, centrally, before the request reaches any service.

- **Runtime Scenario 2 -User Login**

- This scenario demonstrates the JWT authentication flow across the full stack -from credential submission to role-based navigation. It is architecturally relevant because JWT statelessness is a fundamental cross-cutting concern affecting all services.



- **What the diagram shows:**

- The login request bypasses JWT validation at the API Gateway -it is the one endpoint that does not require a token. After successful BCrypt verification, a JWT is issued and stored in Angular, where it is automatically attached to all subsequent requests.

Step-by-step description:

Step	From	To	Action
1	Lecturer	Angular Frontend	Enters username and password in the login form

2	Angular Frontend	API Gateway	Sends POST /auth/login via HTTPS
3	API Gateway	user-service	Forwards request without JWT validation -login endpoint is public
4	user-service	PostgreSQL	Loads user record and BCrypt hash from user_schema
5	user-service	user-service	BCrypt verifies the submitted password against the stored hash (cost factor 12)
alt	—	—	If password is wrong → return HTTP 401 Unauthorized
6	user-service	Angular Frontend	Issues signed JWT containing userId, role ROLE_PROFESSOR and 15-minute expiry -returns HTTP 200 OK
7	Angular Frontend	Angular Frontend	Stores JWT -HttpInterceptor now automatically attaches Authorization: Bearer to all future requests
8	Angular Frontend	Lecturer	Route Guard reads role from JWT -navigates to Lecturer Dashboard

Notable architectural aspects:

- **Step 3 -Public login endpoint:** The /auth/login endpoint is the only endpoint that bypasses JWT validation at the API Gateway. All other endpoints require a valid token.
- **Step 5 -BCrypt security:** BCrypt with cost factor 12 is intentionally slow -this makes brute-force password attacks computationally expensive and impractical.
- **Step 6 -Stateless JWT:** The token contains everything needed for authorization -userId and role. No session state is stored on the server. Any service in the system can independently validate the token without calling user-service again. This is what makes the microservice architecture scalable.
- **Step 7 -HttpInterceptor:** Once the JWT is stored, Angular's HttpInterceptor handles token attachment automatically and consistently across every request. No individual component needs to manage this manually.

7. Deployment View

Infrastructure Level 1

Overview Diagram:-



- **Motivation**
- The University Management System runs as a distributed system across multiple nodes. The architecture separates the presentation layer (Web Server), business logic layer (Application Server with microservices) and data layer (Database Server). External services are connected via standardized REST and HTTPS interfaces.
- The system applies the CAP theorem at service level -critical modules (grades, enrollment, authentication, exam registration) use CP (Consistency + Partition Tolerance). Non-critical modules (announcements, library catalog) use AP (Availability + Partition Tolerance).

Quality and Performance Features

Feature	Description
Consistency (CP)	Critical data is always consistent across all nodes -no stale grades or duplicate enrollments

Partition Tolerance	System continues operating if network connection between nodes is interrupted
Scalability	Each microservice scales independently -enrollment service can scale during registration peak
Fault Isolation	If one service fails, all others continue running -no single point of failure
Schema Isolation	Each service owns its own PostgreSQL schema -no direct cross-service database access
External Integration	External services (SendGrid, Stripe, Moodle) connected via REST API and HTTPS

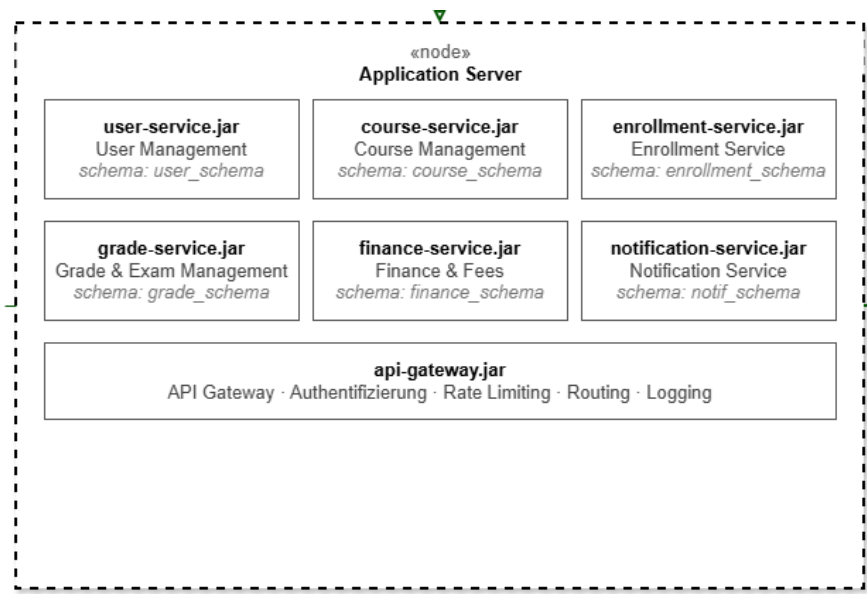
Mapping of Building Blocks to Infrastructure

Software Component	Infrastructure Node	Protocol
Browser	Client Device	—
university-ui.war	Web Server	HTTPS
user-service.jar	Application Server	REST API
course-service.jar	Application Server	REST API
enrollment-service.jar	Application Server	REST API
grade-service.jar	Application Server	REST API
finance-service.jar	Application Server	REST API
notification-service.jar	Application Server	REST API
api-gateway.jar	Application Server	REST API
PostgreSQL :: UniversityDB	Database Server	JDBC/SQL
SendGrid REST API	E-Mail Service (external)	REST API
Stripe API	Payment Gateway (external)	HTTPS

Moodle REST API	Moodle LMS (external)	REST API
-----------------	-----------------------	----------

Infrastructure Level 2

Application Server -Internal Structure:-



Service	Responsibility	Schema	CAP
user-service.jar	User management and authentication	user_schema	CP
course-service.jar	Course management	course_schema	CP
enrollment-service.jar	Course enrollment with capacity enforcement	enrollment_schema	CP
grade-service.jar	Grade and exam management	grade_schema	CP
finance-service.jar	Fee management and payments	finance_schema	CP
notification-service.jar	Email and push notifications	notif_schema	CP
api-gateway.jar	Routing, authentication, rate limiting, logging	—	—

Database Server - Internal Structure:-

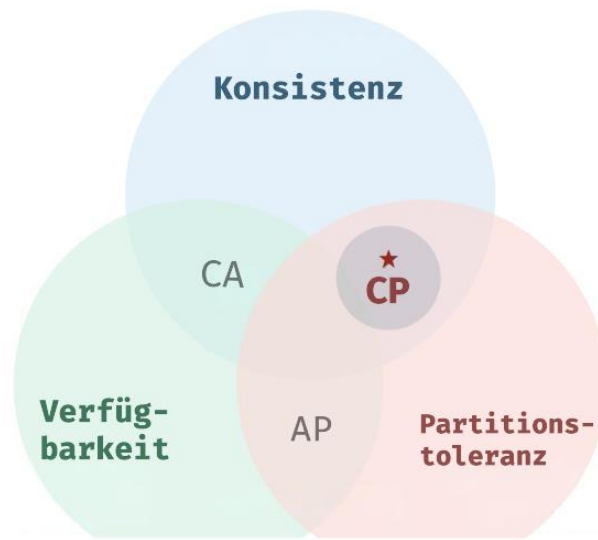
- The system uses the Schema-per-Service pattern. All services share one PostgreSQL instance but each service owns a completely separate schema. No service may access another service's schema directly.

Schema	Owned By	Contents
user_schema	user-service	User accounts, roles, passwords
course_schema	course-service	Course data, descriptions, capacity
enrollment_schema	enrollment-service	Enrollments, seat counts
grade_schema	grade-service	Grades, exam results
finance_schema	finance-service	Fees, payment status
notif_schema	notification-service	Notification logs

External Services -Integration:-

Service	Provider	Protocol	Reason for External Use
E-Mail Service	SendGrid REST API	REST API	Professional cloud provider -no SMTP server required
Payment Gateway	Stripe API	HTTPS	PCI-DSS certification required -legally complex to self-host
Learning Platform	Moodle REST API	REST API	Operated separately by university -integrated via REST

CAP Theorem Analysis



Module	CAP Choice	Reasoning
Grade Management	CP	Incorrect grades are a legal and academic risk - strong consistency is mandatory
Course Enrollment	CP	Double booking is not acceptable - atomic transactions required
Authentication & Access	CP	Outdated access rights are a security risk - consistency takes priority
Exam Registration	CP	Capacity limits must be enforced exactly - no duplicate registrations allowed
Fee & Payment	CP	Payment transactions are financially and legally binding - no inconsistency tolerated
Notifications	CP	A delayed email is acceptable - availability preferred over strict consistency

- The system prioritizes **CP -Consistency and Partition Tolerance** as the overall strategy because the most critical data (grades, enrollments, access rights) is legally binding.
- CA was excluded because network failures are inevitable in distributed systems -partition tolerance is therefore mandatory.

8. Cross-cutting Concepts

Cross-cutting concepts are architectural concerns that affect multiple components of the system simultaneously. They form the basis for conceptual integrity and consistency across the entire University Management Website. Rather than repeating these concerns in every building block description, they are specified centrally here.

Security & Authentication

Why cross-cutting:

- Authentication and authorization affect every layer of the system -the Angular frontend, the Spring Boot backend, and the PostgreSQL database simultaneously. No single building block owns this concern.

Concept:

- All communication between client and server is encrypted using HTTPS (TLS 1.2 minimum). After successful login, Spring Boot issues a signed JWT token compliant with RFC 7519. The token contains the user's ID and role (*ROLE_STUDENT*, *ROLE_PROFESSOR*, *ROLE_ADMIN*) and has a defined expiry time to limit exposure in case of token theft.
- On the Angular side, an `HttpInterceptor` automatically attaches the JWT to the Authorization: Bearer header of every outgoing API request. Route Guards (`CanActivate`) prevent unauthorized navigation at the frontend routing level.
- On the Spring Boot side, a `JwtAuthFilter` intercepts every incoming request, validates the token signature and expiry, and loads the authenticated user into the `SecurityContext`. Method-level authorization is enforced using `@PreAuthorize` annotations:

```
@PreAuthorize("hasRole('PROFESSOR')")
@PostMapping("/grades/{studentId}")
public ResponseEntity<Void> updateGrade(
    @PathVariable Long studentId,
    @RequestBody GradeDTO grade) {
    gradeService.update(studentId, grade);
    return ResponseEntity.ok().build();
}
```

- Passwords are never stored in plain text. BCrypt with a cost factor of 12 is used for password hashing. All API inputs are validated and sanitized server-side to prevent SQL injection and XSS attacks.
- **Affected components:** Angular (`HttpInterceptor`, Route Guards), Spring Boot (`JwtAuthFilter`, `SecurityConfig`, `@PreAuthorize`), PostgreSQL (Row-Level Security)

GDPR & Data Privacy

Why cross-cutting:

- The system processes personal data as defined under GDPR Article 4 -including student names, email addresses, matriculation numbers, grades and payment records. Compliance cannot be achieved by a single component; it requires coordinated measures across the database, backend and all external interfaces.

Concept:

- Personal data fields in PostgreSQL are encrypted at column level using the pgcrypto extension. This ensures that even direct database access does not expose readable personal information.
- Every access to personal data is recorded by a Spring AOP aspect that writes a structured entry to a dedicated audit_log table, satisfying the records of processing activities requirement under GDPR Article 30:

```
@Aspect
@Component
public class DataAccessAuditAspect {
    @Before("execution(* com.univmgmt.repository.*.*(..))")
    public void logDataAccess(JoinPoint jp) {
        log.info("Data access: {} by user: {}",
            jp.getSignature(), getCurrentUser());
    }
}
```

- The right to erasure (GDPR Article 17) is implemented as a dedicated API endpoint. Rather than physically deleting student records -which would break referential integrity with linked grade and payment entries -the endpoint anonymizes the record by replacing personal fields with neutral placeholder values:

```
@DeleteMapping("/students/{id}/personal-data")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> anonymizeStudent(@PathVariable Long id) {
    studentService.anonymize(id);
    auditLogger.log("GDPR_ERASURE", id, getCurrentUser());
    return ResponseEntity.noContent().build();
}
```

- External systems such as the payment provider and government database receive only the minimum data required for their function -no full student profiles are transmitted.
- **Affected components:** PostgreSQL (pgcrypto, audit_log table), Spring Boot (AOP audit aspect, anonymization endpoint), all external API integrations

Error Handling & Logging

Why cross-cutting:

- Errors can originate at any layer -Angular form validation, Spring Boot service logic, or PostgreSQL transaction failures. Without a unified strategy, error formats become inconsistent and production debugging becomes extremely difficult.

Concept:

- A single `@ControllerAdvice` class in Spring Boot intercepts all uncaught exceptions system-wide and maps them to consistent JSON error responses. Each response includes an HTTP status code, an application-specific error code, a human-readable message, and a correlation ID that links the user-facing error to the corresponding server log entry:

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        EntityNotFoundException ex) {
        String correlationId = UUID.randomUUID().toString();
        log.warn("[{}] Entity not found: {}", correlationId, ex.getMessage());
        return ResponseEntity.status(404)
            .body(new ErrorResponse(404, "NOT_FOUND",
                ex.getMessage(), correlationId));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneral(Exception ex) {
        String correlationId = UUID.randomUUID().toString();
        log.error("[{}] Unhandled exception", correlationId, ex);
        alertService.sendCriticalAlert(correlationId, ex);
        return ResponseEntity.status(500)
            .body(new ErrorResponse(500, "INTERNAL_ERROR",
                "An unexpected error occurred.", correlationId));
    }
}
```

- Logging is handled by SLF4J with Logback, configured to write structured JSON log entries in production. Log levels are enforced per environment -INFO in production, DEBUG in development. Critical errors automatically trigger an email notification via the system's email service.
- On the Angular side, a global ErrorHandler catches unhandled frontend exceptions and forwards them to a `/api/client-errors` endpoint, ensuring frontend crashes appear in the same log pipeline as backend errors.
- **Affected components:** Angular (Global ErrorHandler), Spring Boot (`@ControllerAdvice`, Logback, `alertService`), Email Service, PostgreSQL (transaction rollback on error)

Data Validation

Why cross-cutting:

- Invalid data can enter the system from any user-facing form -course registration, grade entry, payment submission. Validation must occur at multiple layers for both user experience and security.

Concept:

- Frontend validation in Angular uses Reactive Forms with built-in and custom validators. This provides immediate feedback to the user and reduces unnecessary API calls. However, frontend validation alone is never sufficient from a security perspective -it can be bypassed by any HTTP client.
- All validation rules are therefore enforced independently on the backend using Spring Boot's Bean Validation (@Valid, @NotNull, @Size, @Pattern). Business rule constraints -such as preventing duplicate course enrollments or grade values outside the permitted range -are enforced at the service layer, not just the database layer.
- **Affected components:** Angular (Reactive Forms, custom validators), Spring Boot (Bean Validation, service-layer constraints), PostgreSQL (database-level constraints as final safety net)

Internationalization (i18n)

Why cross-cutting:

- Language and locale settings affect every user-facing string, date format, number format and error message across the entire frontend.

Concept:

- The primary language of the UI is English. The Angular application is structured from the beginning to support internationalization using Angular's built-in i18n mechanism. All user-facing strings are marked with the i18n attribute, making them extractable for translation without code changes:

```
<h1 i18n>University Management Portal</h1>
<p i18n>Welcome, {{ username }}</p>
```

- German language support is planned as a future extension. Date and time formatting follows the user's locale settings via Angular's DatePipe.
- **Affected components:** Angular (i18n markup, DatePipe, locale configuration)

Testing Strategy

Why cross-cutting:

- Quality assurance is not the responsibility of a single component. Every layer of the system requires its own testing approach to ensure correctness, reliability and regression safety.

Concept:

Level	Scope	Tools
Unit tests	Spring Boot service and utility classes	JUnit 5, Mockito
Integration tests	REST API endpoints with real database	Spring Boot Test, Testcontainers
Frontend unit tests	Angular components and services	Jasmine, Karma
Manual UI testing	Key user flows (enrollment, grade entry, login)	Browser-based

- Test coverage reports are generated automatically and committed to the repository. Every pull request must pass the full test suite before merging.
- **Affected components:** All layers -Angular, Spring Boot, PostgreSQL

9. Architecture Decisions

ADR-01: Three-Tier Architecture

Field	Content
Status	Accepted
Context	The team needed a high-level architectural pattern that cleanly separates user interface concerns, business logic, and data persistence. The system must support three distinct user roles with different views and workflows, and must integrate with external systems such as a payment provider and a learning management system.
Decision	We adopt a classic three-tier architecture: Presentation tier (Angular frontend), Application Logic tier (Spring Boot REST API), and Data tier (PostgreSQL database).
Rejected Alternative	Single-service monolith -rejected because it cannot scale individual features independently and requires a full system redeployment for every change. During enrollment peaks this would make targeted scaling impossible.
Rejected Alternative	Two-tier architecture (frontend directly accessing database) -rejected because it exposes the data tier directly to the client, violating security requirements and GDPR constraints.

Consequences	Clear separation of concerns -each tier can be developed, tested and deployed independently. External system integrations are handled exclusively in the application tier, keeping the frontend clean. The data tier is never exposed directly to the frontend. Introduces network communication overhead between tiers. Requires disciplined API contract management between frontend and backend teams.
---------------------	---

ADR-02: Angular as Frontend Framework

Field	Content
Status	Accepted
Context	The frontend must serve three user groups -students, professors and admins -each with completely different views, permissions and workflows. Complex forms are required for course registration, grade entry and payment processing. Role-based navigation control is essential.
Decision	We use Angular as the frontend framework, with TypeScript, Reactive Forms, Route Guards and HttpInterceptor for JWT handling.
Rejected Alternative	React -rejected because it lacks built-in Route Guards and requires additional third-party libraries for role-based navigation that Angular provides natively. Managing JWT attachment across all requests would require manual implementation rather than a single HttpInterceptor.
Rejected Alternative	Vue.js -rejected because the team has no existing Vue experience, and Vue's ecosystem for enterprise-scale role-based access control is less mature than Angular's built-in module system.
Consequences	Built-in module system enforces clean separation between role-specific feature modules. Route Guards block unauthorized navigation at the client level without additional libraries. Reactive Forms with validators handle complex university-specific input rules. Angular's HttpInterceptor makes JWT attachment and 401/403 handling consistent system-wide. Larger initial bundle size than lighter alternatives - mitigated by lazy loading feature modules per role. Steeper learning curve for developers new to Angular's opinionated structure.

ADR-03: Java Spring Boot as Backend Framework

Field	Content
Status	Accepted

Context	The backend must expose stable REST endpoints consumed by Angular, integrate with three external systems, enforce GDPR compliance, and provide transactional guarantees for grade and payment operations. The development team has existing Java experience and the organization runs Java services in production.
Decision	We use Java Spring Boot with a layered REST architecture (Controller → Service → Repository) as the backend framework.
Rejected Alternative	Node.js / Express -rejected because it lacks native transactional guarantees (@Transactional) and requires additional libraries for security. Spring Security provides direct JWT integration without third-party dependencies. Node.js also has a weaker ecosystem for enterprise-grade ORM compared to Spring Data JPA with Hibernate.
Rejected Alternative	Python / FastAPI -rejected because the team has no existing Python expertise in a production context, and FastAPI lacks the mature enterprise integration ecosystem that Spring Boot provides for GDPR-compliant data handling and connection pooling.
Consequences	Spring Security integrates directly with our JWT authentication strategy -no third-party auth library required. @Transactional guarantees atomicity for critical operations such as simultaneous grade updates and payment recording. Spring Profiles allow seamless switching between local development and production configuration. Spring Data JPA with Hibernate maps the relational university data model without raw SQL. Higher memory footprint than lighter alternatives such as Node.js -acceptable within the project's infrastructure budget. Longer cold start time - not relevant as the backend runs as a persistent service rather than serverless functions.

ADR-04: PostgreSQL as Primary Database

Field	Content
Status	Accepted
Context	University data -students, courses, enrollments, grades, payments -is highly structured with complex many-to-many and one-to-many relationships. GDPR compliance requires column-level encryption, audit logging and the ability to anonymize individual records on request. The tech stack constraint TC_1 specifies PostgreSQL as the required database technology.
Decision	We use PostgreSQL as the sole primary database, accessed via Spring Data JPA with Hibernate as the ORM layer.
Rejected Alternative	MongoDB (NoSQL) -rejected because university data has a highly structured relational model with foreign key constraints between students, courses, grades and payments.

	MongoDB's flexible schema provides no benefit here and would require application-level enforcement of referential integrity. Column-level encryption via pgcrypto is not available in MongoDB without a separate encryption service.
Rejected Alternative	MySQL -rejected in favor of PostgreSQL because PostgreSQL provides Row-Level Security policies natively, the pgcrypto extension for GDPR-compliant column encryption, and superior support for complex analytical queries on grade and enrollment data. PostgreSQL was also explicitly specified in constraint TC_1.
Consequences	Foreign key constraints enforce referential integrity at the database level -grades and payments cannot exist without valid student and course records. ACID transactions ensure that course enrollment and payment charging are atomic -either both succeed or neither does. Row-Level Security policies restrict professors to reading only grades from their own courses, enforced independently of application logic. The pgcrypto extension provides column-level encryption for GDPR-sensitive fields without a separate encryption service. Schema changes require Flyway migration scripts -adds discipline overhead but prevents environment inconsistencies. Less flexible than NoSQL solutions for unstructured data -not a concern given the highly structured nature of university data.

10. Quality Requirements

10.1 Quality Requirements Overview

The top five quality goals are already defined in Section 1.2 with concrete scenarios. This section captures additional quality requirements of lower priority that are desirable but not critical.

Category	Quality Requirement	Priority
Reliability	System must recover automatically from a single service failure without manual intervention	High
Security	All API endpoints must enforce role-based access -no endpoint accessible without a valid JWT	High
Performance Efficiency	Page load time must not exceed 3 seconds under peak load for 95% of requests	Medium
Maintainability	A new developer must be able to understand and modify a single service within one working day	Medium
Compatibility	System must work correctly on Chrome, Firefox and Safari -latest two versions each	Medium
Usability	All core workflows (enrollment, grade entry, login) must be completable in fewer than 5 clicks	Low
Flexibility	Adding a new external service provider must require changes to only one service	Low

10.2 Quality Scenarios

- Scenario 1 -Double Enrollment Prevention

Field	Content
Scenario ID	QS-01
Scenario Name	Concurrent Enrollment Conflict
Source	Two students simultaneously
Stimulus	Both submit enrollment request for the last available seat in the same course
Environment	Normal operation -enrollment period peak load
Artifact	enrollment-service -EnrollmentService -PostgreSQL enrollment_schema
Response	Exactly one student receives enrollment confirmation -the other receives a clear conflict error
Response Measure	Zero cases of double-booking across 10,000 concurrent enrollment test requests

- Scenario 2 -Service Failure Isolation

Field	Content
Scenario ID	QS-02
Scenario Name	notification-service Outage
Source	Infrastructure failure
Stimulus	notification-service crashes during an enrollment peak
Environment	Production -enrollment period
Artifact	notification-service -RabbitMQ queue -enrollment-service
Response	Enrollments continue to succeed -undelivered notifications remain in the RabbitMQ queue and are delivered when notification-service recovers
Response Measure	Zero enrollment failures caused by notification-service outage -100% of queued notifications delivered after recovery

- Scenario 3 -Unauthorized Access Attempt

Field	Content
Scenario ID	QS-03
Scenario Name	Role Violation Attempt
Source	Student user
Stimulus	Student sends a request to PUT /grades/{id} -an endpoint restricted to lecturers
Environment	Normal operation
Artifact	API Gateway -JwtAuthFilter -grade-service
Response	Request is rejected at the API Gateway with HTTP 403 Forbidden -grade-service never receives the request
Response Measure	100% of unauthorized role-violation attempts rejected before reaching the target service

- Scenario 4 -Peak Load Performance

Field	Content
Scenario ID	QS-04
Scenario Name	Enrollment Period Load
Source	500 concurrent students
Stimulus	500 students simultaneously submit enrollment requests at semester start
Environment	Production -enrollment period peak
Artifact	enrollment-service -PostgreSQL -API Gateway
Response	enrollment-service scales horizontally -all requests processed within acceptable time
Response Measure	95% of requests receive a response within 3 seconds -no requests time out

11. Risks and Technical Debts

Risks are ordered by priority. Each risk includes a mitigation strategy. Technical debts are tracked in the product backlog and addressed at a rate of 10–20% per sprint.

- Risks

Priority	Risk	Probability	Impact	Mitigation
1	Optimistic locking retry storms - Under extreme concurrency during enrollment peak, high version-conflict rates could degrade performance	Medium	High	Implement exponential backoff retry logic in enrollment-service -load test before each enrollment period
2	External service dependency - SendGrid or Stripe API outage could impact email delivery and payment processing	Medium	Medium	RabbitMQ queue buffers notifications -payments fail gracefully with clear user error message and retry option
3	GDPR non-compliance risk - Missing anonymization of a personal data field could result in regulatory violation	Low	Very High	Automated GDPR audit test verifies all personal fields are covered by anonymization endpoint before each release
4	Schema migration failures - Flyway migration script error could leave database in inconsistent state during deployment	Low	High	All migrations are tested in staging environment before production deployment - rollback scripts prepared for every migration
5	JWT token theft -A stolen JWT token could allow unauthorized access until token expiry	Low	High	Short token expiry time (15 minutes) with refresh token pattern -HTTPS enforced on all endpoints

- Technical Debts

ID	Description	Impact	Plan
TD-01	notification-service currently has no dead-letter queue for failed SendGrid deliveries	Failed notifications are silently lost	Implement dead-letter queue in Sprint 4
TD-02	API Gateway does not yet implement circuit breaker pattern	Slow downstream service can block API Gateway threads	Add Resilience4j circuit breaker in Sprint 5
TD-03	No centralized distributed tracing across microservices	Debugging cross-service issues requires manual log correlation	Integrate OpenTelemetry in Sprint 6
TD-04	finance-service has no automated integration test for Stripe webhook handling	Payment confirmation edge cases not covered	Add Stripe webhook test suite in Sprint 5

12. Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability - the four properties that guarantee reliable database transactions
API Gateway	A single entry point that routes all incoming requests to the appropriate microservice -also handles authentication and rate limiting
CAP Theorem	A principle stating that a distributed system can only guarantee two of three properties simultaneously: Consistency, Availability and Partition Tolerance
CP	Consistency + Partition Tolerance -the CAP strategy chosen for critical modules. The system prefers to return an error rather than show incorrect data
AP	Availability + Partition Tolerance -the CAP strategy for non-critical modules. The system remains reachable even if data is briefly outdated
JWT	JSON Web Token - a signed, stateless token used for authentication. Contains the user's ID and role and is validated by every service independently
Microservice	A small, independently deployable service that is responsible for exactly one business capability and owns its own data schema
Optimistic Locking	A concurrency control strategy where conflicts are detected at commit time using a version number -no lock is held during the transaction
Pessimistic Locking	A concurrency control strategy where a row is locked before it is read -prevents any other transaction from modifying it until the lock is released
Schema-per-Service	A database design pattern where each microservice owns a dedicated schema within a shared database -no direct cross-schema access is permitted
GDPR	General Data Protection Regulation -EU law that governs how personal data must be collected, stored, processed and deleted
BCrypt	A password hashing algorithm that is intentionally slow to resist brute-force attacks -used with cost factor 12 in this system
RabbitMQ	A message broker that enables asynchronous communication between services using the AMQP protocol
Enrollment	The process by which a student registers for a university course -subject to capacity limits and prerequisite checks
Partition Tolerance	The ability of a distributed system to continue operating correctly even when network communication between nodes is interrupted
Technical Debt	The additional work created by choosing a quick or easy solution now instead of a better, more sustainable approach

Flyway	A database migration tool that tracks and applies schema changes in a controlled and versioned way
Three-tier Architecture	An architectural pattern that separates a system into three layers: Presentation (Angular), Application Logic (Spring Boot) and Data (PostgreSQL)